# QAOA-in-QAOA: solving large-scale MaxCut problems on small quantum machines

Zeqiao Zhou,[1,2,*] Yuxuan Du,[2,†] Xinmei Tian,[1,‡] and Dacheng Tao[2,§]

[1] *Department of Electronic Engineering and Information Science, University of Science and Technology of China, Hefei, Anhui, China, 230027*

[2] *JD Explore Academy, Beijing 101111, China*

(Dated: May 25, 2022)

The design of fast algorithms for combinatorial optimization greatly contributes to a plethora of domains such as logistics, finance, and chemistry. Quantum approximate optimization algorithms (QAOAs), which utilize the power of quantum machines and inherit the spirit of adiabatic evolution, are novel approaches to tackle combinatorial problems with potential runtime speedups. However, hurdled by the limited quantum resources nowadays, QAOAs are infeasible to manipulate large-scale problems. To address this issue, here we revisit the MaxCut problem via the divide-and-conquer heuristic: seek the solutions of subgraphs in parallel and then merge these solutions to obtain the global solution. Due to the $\mathbb{Z}_2$ symmetry in MaxCut, we prove that the merging process can be further cast into a new MaxCut problem and thus be addressed by QAOAs or other MaxCut solvers. With this regard, we propose QAOA-in-QAOA (QAOA$^2$) to solve arbitrary large-scale MaxCut problems using small quantum machines. We also prove that the approximation ratio of QAOA$^2$ is lower bounded by 1/2. Experiment results illustrate that under different graph settings, QAOA$^2$ attains a competitive or even better performance over the best known classical algorithms when the node count is around 2000. Our method can be seamlessly embedded into other advanced strategies to enhance the capability of QAOAs in large-scale combinatorial optimization problems.

arXiv:2205.11762v1 [quant-ph] 24 May 2022

## I. INTRODUCTION

Combinatorial optimization [1], which aims to search for maxima or minima of an objective function with discrete solution space, is an indispensable tool in various application domains such as portfolio investment, vehicle routing and transportation [2, 3]. Driven by its fundamental importance, huge efforts have been dedicated to devising efficient algorithms for combinatorial problems during past decades. Representative examples include using semidefinite programming techniques to approximate the solution of MaxCut and maximum 2-satisfiability problems [4, 5], adopting the simulated annealing methods to solve constrained problems [6], and exploiting other heuristics such as expert knowledge and the structure of studied problems to solve traveling salesman problems [7–9]. Recently, attributed to the power of neural networks, deep learning techniques have also been employed in solving combinatorial optimization problems [10–12]. Nevertheless, due to the intrinsic hardness of most combinatorial problems [13], existing methods request expensive computational overhead to estimate the optimal solution and thus it is highly demanded to design more powerful algorithms to speed up the optimization.

Quantum computers have the ability to efficiently solve certain problems that are computationally hard for classical computers [14]. This superiority could be preserved for noisy intermediate-scale quantum (NISQ) machines [15], because both theoretical and experimental studies have exhibited their runtime merits over the classical counterparts for certain tasks [16–18]. For this reason, there is a growing interest in designing NISQ algorithms with computational merits. Variational quantum algorithms (VQAs) [19], which consist of parameterized quantum circuits [20] and classical optimizers to adequately leverage accessible quantum resources and suppress the system noise, are leading solutions to achieve this goal. Notably, initial studies have exhibited that quantum approximate optimization algorithms (QAOAs) [21], as one crucial paradigm of VQAs, can be used to tackle combinatorial optimization with potential computational advantages [22, 23]. The underlying principle of QAOAs is mapping a quadratic unconstrained binary optimization (QUBO) problem describing the explored combinatorial problem to a Hamiltonian whose ground state refers to the optimal solution [24, 25]. In this way, various manipulable quantum systems can be used to advance combinatorial problems [26–28].

Envisioned by the promising prospects, both empirical and theoretical studies have been carried out to understand the foundation of QAOAs and improve their performance. One critical line of research is unveiling the connection between adiabatic quantum computation [29, 30] and QAOAs and showing that QAOAs can be seen as a parameterized Trotterization of adiabatic evolution [21, 31, 32]. Making use of this relation, the parameter initialization of QAOAs can be simplified associated with an improved performance [32–34]. In parallel to explore the initialization strategy, another crucial topic is designing advanced training strategies of QAOA to avoid local optima and accelerate optimization. Concrete examples include modifying the objective functions [35], applying the iterative training strategy [36, 37], and

* This work was done when he was a research intern at JD Explore Academy, zhouzeqiao@mail.ustc.edu.cn

† Corresponding author, duyuxuan123@gmail.com

‡ Corresponding author, xinmei@ustc.edu.cn

§ Corresponding author, dacheng.tao@sydney.edu.au

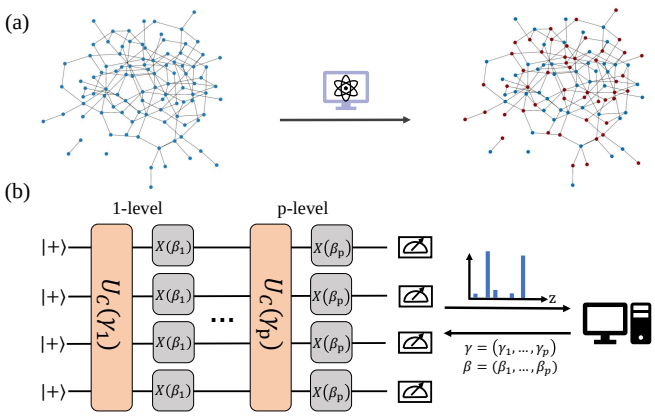


FIG. 1. **MaxCut and QAOA** (a) One instance of MaxCut with 100 vertices. The left part is the problem graph. In the right part, two subsets of vertices are distinguished by its color as blue or red. (b) For a $p$-level QAOA, $U_C(\gamma_i)$ and $U_B(\beta_i)$ are alternately applied to an initial state. The classical optimizer uses the measured bitstring to updated parameters of the circuit.

using adaptive mixing operators [34, 38–40]. Despite the remarkable achievements, little progress has been made in overcoming the scalability issue of QAOAs, whereas the ultimate goal of the most advanced QAOA is solving a problem with hundreds of vertices [41]. The main challenges come from the fact that manipulating a graph with $n$-nodes requires $O(n)$ qubits but the most advanced quantum machines nowadays can only provide a very limited number of qubits with $n \approx 100$. Moreover, due to the a high-level noise and barren plateaus, QAOAs may suffer from the trainability issue for the large $n$ [42–46], which degrade their performance [28, 47]. Although an initial attempt of the scalable QAOAs has been addressed by [48, 49], their approach encounters the sample complexity issue [50]. To this end, it still remains obscure whether QAOAs can outperform classical approaches towards large-scale combinatorial problems.

To further enhance the capability of QAOAs, here we investigate the possibility of QAOAs toward solving large-scale MaxCut problems, especially when the problem size is greatly larger than the qubit count of the accessible quantum machines. In particular, we revisit MaxCut through the lens of the divide-and-conquer heuristic, i.e., splitting the given graph into multiple subgraphs, seeking the solutions of these subgraphs in parallel, and then merging these solutions to obtain the global solution. Notably, we prove that due to the inherent $\mathbb{Z}_2$ symmetry in MaxCut, the merging process can be cast to a new MaxCut problem. This observation hints that the large-scale MaxCut problem can be tackled by MaxCut solvers in a hierarchical way. To this end, we propose QAOA-in-QAOA (QAOA$^2$) to solve MaxCut with tens of thousands of nodes using NISQ machines. In addition, QAOA$^2$ can integrate with the community detection method in the process of graph partition to attain better performance. On the theoretical side, we show that the lower bound of the approximation ratio of QAOA$^2$ is 1/2. On the experimental side, we first approximate the solution of MaxCut instances with 2000 vertices using QAOA$^2$ executed on a 10-qubit quantum simulator. The achieved results are competitive or even better than the best known classical method. Moreover, through a systematical investigation, we verify that the density of graphs and subgraphs, the size of subgraphs and partition strategies are the decisive factors effecting the performance of QAOA$^2$. These results suggest that QAOA$^2$ provides a potential and novel way of utilizing NISQ machines to solve practical learning problems with computational advantages.

## II. PRELIMINARIES

The main focus of this study is solving MaxCut problems as shown in Fig. 1(a). Formally, let $\mathsf{G}(\mathsf{V}, \mathsf{E})$ be an undirected graph, where the number of vertices is $|\mathsf{V}| = N$ and the edge weight for $(i, j) \in \mathsf{E}$ is $W_{ij} = W_{ji}$. Define a cut as a partition of the original set $\mathsf{V}$ into two subsets $\mathsf{S}$ and $\mathsf{T}$ with $\mathsf{V} = \mathsf{S} \bigcup \mathsf{T}$ and $\mathsf{S} \bigcap \mathsf{T} = \emptyset$. The aim of MaxCut is to find $\mathsf{S}$ maximizing the sum of edge weight connecting vertices in $\mathsf{S}$ and $\mathsf{T}$. Denote $\boldsymbol{z}_i = +1$ ($\boldsymbol{z}_i = -1$) when the $i$-th vertex is in $\mathsf{S}$ ($\mathsf{T}$), any partition of $\mathsf{V}$ can be represented by a bitstring $\boldsymbol{z} \in \{+1, -1\}^n$. The optimal solution $\boldsymbol{z}^*$ of MaxCut maximizes the following objective function

$$C(\boldsymbol{z}) := \frac{1}{2} \sum_{(i,j)\in\mathsf{E}} W_{ij}(1 - \boldsymbol{z}_i \boldsymbol{z}_j) = c - \frac{1}{2} \sum_{(i,j)\in\mathsf{E}} W_{ij} \boldsymbol{z}_i \boldsymbol{z}_j, \tag{1}$$

where $c = \frac{1}{2} \sum_{(i,j)\in\mathsf{E}} W_{ij}$ only depends on the problem and is independent of $\boldsymbol{z}$. Theoretical studies have proven that finding $\boldsymbol{z}^*$ is NP-hard so in most cases we are searching for an approximation of $\boldsymbol{z}^*$ [13]. The best-known classical MaxCut solver is Goemans-Williamson (GW) algorithm [5], which uses the semi-definite programming (SDP) technique to ensure 0.879 approximation ratio [4].

To carry out combinatorial problems on physical systems, it is necessary to map the problem to a Hamiltonian whose ground state corresponds to the optimal solution [25]. In the task of MaxCut with $n$ vertices, this Hamiltonian yields

$$H_C = \sum_{(i,j)\in\mathsf{E}} W_{ij} Z_i Z_j, \tag{2}$$

where $Z_i$ refers to the Pauli-Z operator applied on the $i$-th qubit with $i \in [n]$ [51]. The optimal solution $|\boldsymbol{z}^*\rangle$ amounts to the computational basis state $|\boldsymbol{z}\rangle$ minimizing $\langle \boldsymbol{z}|H_C|\boldsymbol{z}\rangle$. Since $H_C$ is diagonal, $|\boldsymbol{z}^*\rangle$ also refers to its ground state.

Quantum approximate optimization algorithm and its variants (QAOAs) [21], which absorb the merits of quantum annealing [22] and variational quantum algorithms [19], are proposed to solve combinatorial problems

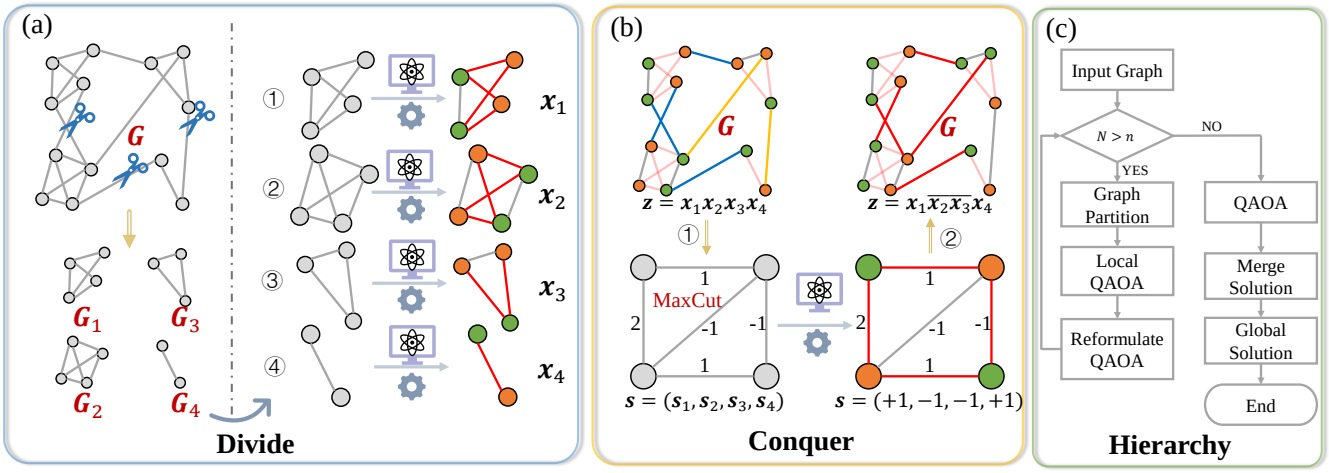


FIG. 2. **Scheme of QAOA$^2$.** (a) A graph is partitioned into four parts, where each one is no larger than the available number of qubits (e.g., 4 qubits). Then the adopted MaxCut solvers are applied to all subgraphs in parallel. The green node refers to the bit $+1$ and the orange node refers to the bit $-1$. The cut edges are in red. (b) The first step highlighted by the brown arrow refers to merge local solutions of all subgraphs and calculate the value of cut between subgraphs. Yellow lines refer to 'cut' and blue lines refer to 'uncut'. The lower left plot indicates that the accommodation of local solutions can be reformulated a new Maxcut problem with four nodes. The first step highlighted by the brown arrow means that MaxCut solvers are applied to solve this new problem. Then, local solutions are merged according to solution of $s$. (c) An extremely large MaxCut can be solved by applying QAOA$^2$ in a hierarchy way. When the graph size is above the limitation of qubits, it is partitioned and solved locally, and then reformed as a new MaxCut until its size is no larger than the available number of qubits. After all optimizations, low-level local solutions are merged according to high-level solutions.

on NISQ machines with potential advantages. As shown in Fig. 1(b), when applied to MaxCut, QAOA approximates the ground state by an ansatz state

$$|\Phi(\boldsymbol{\gamma},\boldsymbol{\beta})\rangle = U_B(\boldsymbol{\beta}_p)U_C(\boldsymbol{\gamma}_p)\dots U_B(\boldsymbol{\beta}_1)U_C(\boldsymbol{\gamma}_1)|s\rangle \quad (3)$$

where $|s\rangle = (|0\rangle^{\otimes n} + |1\rangle^{\otimes n})/\sqrt{2}$ is the initial state, $\boldsymbol{\gamma},\boldsymbol{\beta} \in (0, 2\pi]^p$ are variational parameters, and $U_B(\beta) = \exp(-i\beta\sum_k^n X_k)$ and $U_C(\gamma) = \exp(-i\gamma H_C)$. To find $|\boldsymbol{z}^*\rangle$, a classical optimizer is used to update $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$ by minimizing the following objective function

$$C(\boldsymbol{\gamma},\boldsymbol{\beta}) = \langle\Phi(\boldsymbol{\gamma},\boldsymbol{\beta})|H_C|\Phi(\boldsymbol{\gamma},\boldsymbol{\beta})\rangle. \quad (4)$$

In the optimal setting, the optimal solution yields $|\boldsymbol{z}^*\rangle = |\Phi(\boldsymbol{\gamma}^*,\boldsymbol{\beta}^*)\rangle$ with $|\Phi(\boldsymbol{\gamma}^*,\boldsymbol{\beta}^*)\rangle = \arg\min C(\boldsymbol{\gamma},\boldsymbol{\beta})$.

## III. QAOA-IN-QAOA

Standard QAOAs require $n$ qubits to solve a MaxCut problem with $n$ vertices. This linear dependence suppresses the power of QAOA, since the accessible quantum resources nowadays are vey limited. To assign the capability of QAOAs for solving large-scale problems, here we revisit MaxCut through the lens of the divide-and-conquer heuristic. Specifically, in the *dividing* step, we partition the given graph $\mathsf{G}$ into $h$ subgraphs $\{\mathsf{G}_i(\mathsf{V}_i,\mathsf{E}_i)\}_{i=1}^h$, where $\mathsf{V} = \bigcup_{i=1}^h \mathsf{V}_i$ and $\mathsf{V}_i \bigcap \mathsf{V}_j = \emptyset$ when $i \neq j$. An intuition of this partition process is exhibited in Fig. 2(a). Once the partition is completed, the MaxCut solvers are exploited to seek optimal solutions of these subgraphs in parallel. We denote the optimized solutions for all $h$ subgraphs as $\{\boldsymbol{x}_i\}_{i=1}^h$ with $\boldsymbol{x}_i \in \{+1,-1\}^{|\mathsf{V}_i|}$ for $\forall i \in [h]$. Due to the $\mathbb{Z}_2$ symmetry in MaxCut [36], the bitstring $\bar{\boldsymbol{x}}_i$, which flips all bits in $\boldsymbol{x}_i$, also corresponds to the solution of $\mathsf{G}_i$ for $\forall i \in [h]$. In the *conquering* step, as shown in Fig. 2(b), the obtained solutions of all subgraphs are merged to obtain the global solution $\boldsymbol{z}$ of $\mathsf{G}$. Since there are two solutions for each subgraph, the total number of the possible global solutions is $2^h$, i.e., $\boldsymbol{z} \in \mathcal{Z} := \{\boldsymbol{x}_1,\bar{\boldsymbol{x}}_1\}\oplus\{\boldsymbol{x}_2,\bar{\boldsymbol{x}}_2\}...\oplus\{\boldsymbol{x}_h,\bar{\boldsymbol{x}}_h\}$. Taking into account the connections among $h$ subgraphs, the global solution yields

$$\hat{\boldsymbol{z}} = \arg\max_{\boldsymbol{z}\in\mathcal{Z}} C(\boldsymbol{z}). \quad (5)$$

The following theorem illustrates that seeking $\hat{\boldsymbol{z}}$ can be cast into a new MaxCut problem, where the corresponding proof is provided in Appendix B.

**Theorem 1.** *Suppose that the graph $\mathsf{G}$ is partitioned into $h$ subgraphs $\{\mathsf{G}_i\}_{i=1}^h$ and the optimized local solutions are $\{\boldsymbol{x}_i\}_{i=1}^h$. To find the bitstring $\hat{\boldsymbol{z}}$ in Eq. (5), let $\boldsymbol{s}_i = +1$ (or $-1$) be the indicator of adopting $\boldsymbol{x}_i$ (or $\bar{\boldsymbol{x}}_i$). Then $\hat{\boldsymbol{z}}$ is identified by $\boldsymbol{s} = (\boldsymbol{s}_1,\dots,\boldsymbol{s}_h) \in \{\pm1\}^h$, i.e.,*

$$\max_{\boldsymbol{z}\in\mathcal{Z}} C(\boldsymbol{z}) \equiv \max_{\boldsymbol{s}} C'(\boldsymbol{s}) = -\frac{1}{2}\sum_{i<j} w'_{ij}\boldsymbol{s}_i\boldsymbol{s}_j + c, \quad (6)$$

*where $w'_{ij} = \frac{1}{2}\sum_u\sum_v \boldsymbol{x}_i^{(u)}\boldsymbol{x}_j^{(v)}W_{iu,jv}$, $\boldsymbol{x}_i^{(u)}$ is the $u$-th bit in $\boldsymbol{x}_i$, $W_{iu,jv}$ is the weight of edge between the two nodes*

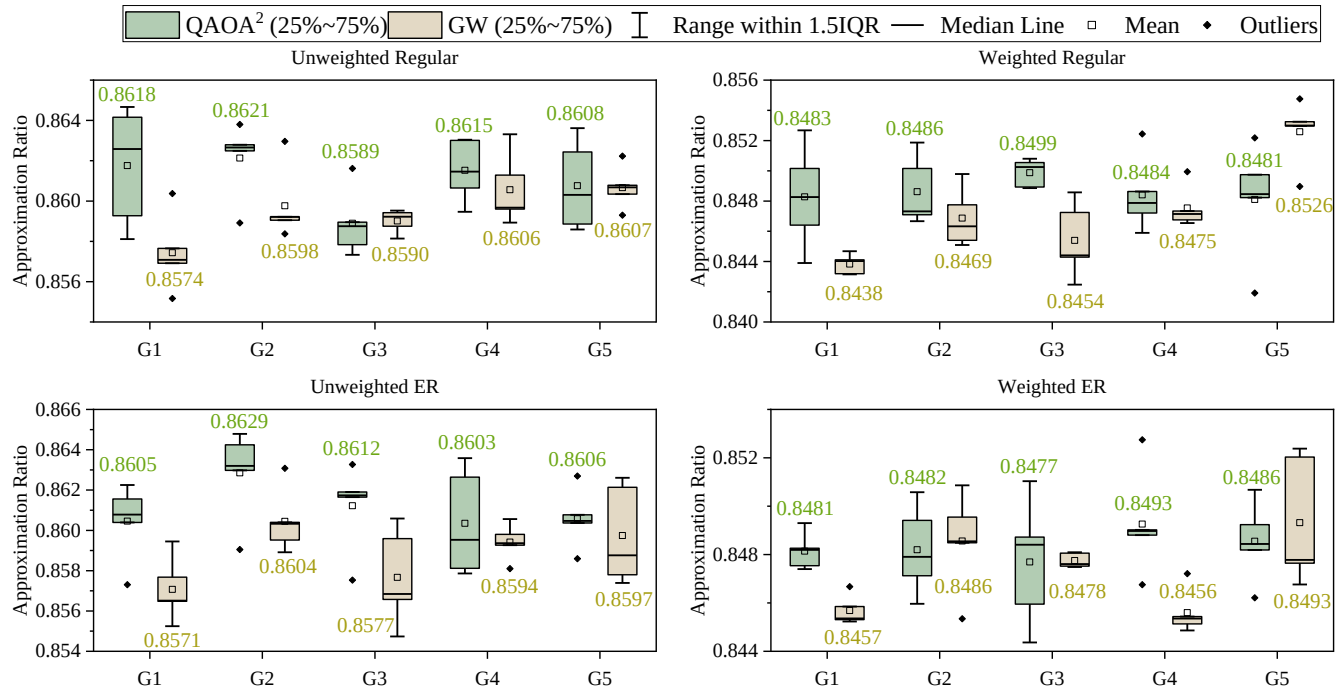


FIG. 3. **Approximation ration of QAOA$^2$ and GW over 2000-vertex graph instances under $C^*_{\mathbf{SDP}}$.** Numerical results on four types of graphs. There are five instances in each type. The number of vertices is fixed to be 2000. The x-axis represents the graph instance G1-G5 and the y-axis represents the approximation ratio. In each panel, QAOA$^2$ is in green and GW is in yellow. The number beside each box is the mean value of approximation ratios in five trials, which is represented by the square symbol in each box.

*corresponding to* $\boldsymbol{x}_i^{(u)}$ *and* $\boldsymbol{x}_j^{(v)}$*, and* $c = \sum_i C_i(\boldsymbol{x}_i) + \sum_{i<j} \frac{1}{2} \sum_u \sum_v W_{iu,jv}$*,* $C_i(\boldsymbol{x}_i)$ *is the optimized value of cut for the subgraph* $\mathsf{G}_i$*.*

The above results hint that MaxCut problems can be tackled in a hierarchical way. According to the reformulated MaxCut in the divide-and-conquer manner, we devise QAOA-in-QAOA, dubbed *QAOA*$^2$, which allows us to use an $n$-qubit quantum machine to solve an $N$-vertex MaxCut with $n \ll N$. The schematic of QAOA$^2$ is shown in Fig. 2. More specifically, in the partition procedure, the graph is divided into several subgraphs that are compatible with quantum devices. This step can be achieved by using random clustering, community detection, or other advanced strategies (See Appendix A for elaborations). The setting of $h$ is flexible, whereas the only requirement is that the size of subgraphs should be less than $n$, i.e., $|\mathsf{V}_i| \leq n$, $\forall i \in [h]$. After partitions, all subgraphs $\{\mathsf{G}_i\}_{i=1}^h$ are solved independently by QAOAs to collect $\{\boldsymbol{x}_i\}_{i=1}^h$. Last, to obtain the estimated global solution $\hat{\boldsymbol{z}}$, we apply QAOAs again to optimize the merging through reformulated MaxCut according to Theorem 1. Note that when $h > n$, the available number of qubits is insufficient to complete the merging process. As such, QAOA$^2$ applies the partition procedure successively on $C'(\boldsymbol{s})$ until the number of subgraphs is no larger than $n$. The diagram of QAOA$^2$ is summarized in Fig. 2(c).

QAOA$^2$ embraces two attractive theoretical advantages. First, compared with other QAOA solvers, QAOA$^2$ is immune to the scalability issue. This characteristic is highly desired for NISQ machines, which provides the opportunity to attain potential quantum advantages towards large-scale combinatorial problems. Moreover, the following theorem whose proof is given in Appendix C guarantees that the approximation ratio of QAOA$^2$ is always better than the random guess.

**Theorem 2.** *Following notations in Theorem 1, the estimated solution* $\hat{\boldsymbol{z}}$ *output by QAOA*$^2$ *always satisfies*

$$C(\hat{\boldsymbol{z}}) \geq \frac{1}{2} \sum_{(i,j)\in\mathsf{E}} W_{ij}. \tag{7}$$

## IV. EXPERIMENT RESULTS

In this section, we conduct numerical simulation to evaluate the ability of QAOA$^2$ towards large-scale MaxCut problems. Specifically, we first give the setup of our experiments, including hyper-parameter settings and the constructions of graphs for MaxCut. Then we compare performance of our proposal with other MaxCut solvers. Last, we comprehend the potential factors influencing the performance of QAOA$^2$.

To fully evaluate the performance of QAOA$^2$, we collect a set of graphs varying in size, degrees of nodes, and

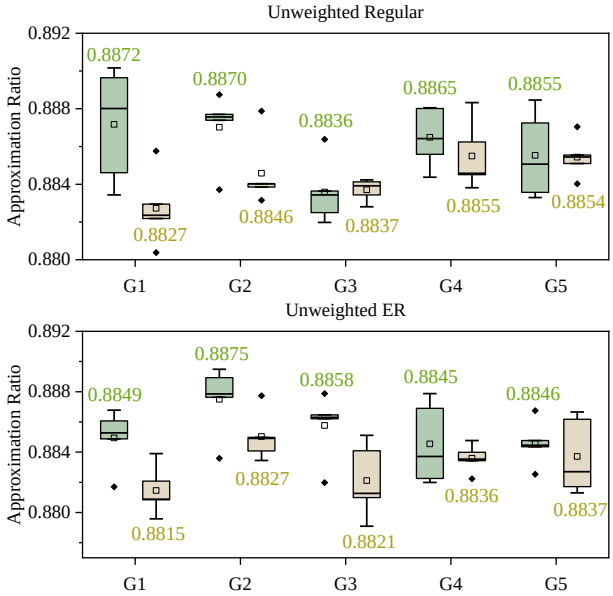


FIG. 4. **Approximation ration of QAOA$^2$ and GW over 2000-vertex graph instances under $C^*_{\text{asymp}}$.** The labels follow the same meaning with those used in Fig. 3. The only difference is the way of calculting the approximation ratio, where the denominator is replaced by $C^*_{\text{asymp}}$.

the weight of edges. For convenience, an unweighted (or weighted) $d$-regular graph with $n$ nodes is abbreviated as "u$d$r-$n$" (or "w$d$r-$n$"). Similarly, for the Erdos-Renyi (ER) graph whose degree of each node is randomly assigned, an unweighted (or weighted) ER graph with the average degree being $d$ and the node size being $n$ is abbreviated as "u$d$e-$n$" (or "w$d$e-$n$"). For the weighted graphs, their weights are integers uniformly sampled from $[0, 5]$. For each setting, we generate multiple instances to obtain the statistical results of the employed MaxCut solver.

When QAOA$^2$ is applied, we specify that the allowable number of qubits is $n \leq 10$ and the random partition strategy is adopted. To find the local solutions $\boldsymbol{x}_i$ in the dividing step, we use a 1-layer QAOA (i.e., $p = 1$ in Eq. (3)) and the number of iterations for training QAOA is set as $T = 20$. To systematically investigate the potential of QAOA$^2$ compared with the classical MaxCut solvers, we employ GW algorithm as the reference. The setting of hyper-parameters of adopted MaxCut solvers is given in Appendix E.

In all simulations, we adopt the approximation ratio as the metric to compare the performance of different MaxCut solvers. Mathematically, given a MaxCut problem, denote $C_A$ as the cuts achieved by algorithm $\mathcal{A}$, the approximation ratio yields

$$r_{\mathcal{A}} = \frac{C_{\mathcal{A}}}{C^*}. \tag{8}$$

where $C^*$ denotes the optimal value of MaxCut. Considering that the exact optimal value $C^*$ is exponentially difficult to get for large $N$, we substitute it with the optimal value $C^*_{\text{SDP}}$ of semidefinite programming optimization step in GW. Another metric we adopt for unweighted regular graphs and unweighted ER graphs with degree $d$ is an asymptotic value derived in [52], where the optimal value of cut is $(\frac{d}{4}+P\sqrt{\frac{d}{4}}+o(\sqrt{d}))N$ and $P = 0.7632$ is the constant in Parisi formula. In calculating approximation rate, we set the optimal result as $C^*_{\text{asymp}} = (\frac{d}{4}+P\sqrt{\frac{d}{4}})N$.

We first apply QAOA$^2$ to 2000-vertex graphs, i.e., u100r-2000, w100r-2000, u100e-2000, and w100e-2000. The numerical results are shown in Fig. 3 and Fig. 4 under the measures $C^*_{\text{SDP}}$ and $C^*_{\text{asymp}}$, respectively. Each setting includes 5 graph instances. Meanwhile, for each instance, we use 5 different random seeds to initialize parameters of QAOA$^2$. In almost all instances, QAOA$^2$ outperforms GW. For example, under the measure $C^*_{\text{SDP}}$, the averaged approximation ratio QAOA$^2$ is higher than that of GW, except for the instance $G_3$ with u1000r-2000, the instance G5 with w1000r-2000, and the instances G2, G3, and G5 with w1000e-2000. Under the measure $C^*_{\text{asymp}}$, the averaged approximation ratio QAOA$^2$ is higher than that of GW, except for the instance $G_3$ with u1000r-2000. In this instances, the maximal differences of the approximation ratio is only 0.0001. Moreover, both QAOA$^2$ and GW can obtain a better performance on unweighted graphs than weighted graphs. A possible reason is the intrinsic hardness in finding optimal solution for weighted graphs. Besides, since the distribution of weighted edges is unbalanced and the graph connectivity is sparse, random partition used in QAOA$^2$ is not a suitable choice, which may leave most edges remained among subgraphs. To this end, we exhibit how advanced partition methods, i.e., community detection algorithm, can further improve the power of QAOA$^2$ in Appendix E. Note that QAOA$^2$ may attain a better runtime efficiency than GW, benefiting from its hierarchical scheme and the computational advantage of quantum algorithm.

We next explore the potential factors that may influence the performance of QAOA$^2$. As mentioned previously, the sparsity of graph may reduce the power of QAOA$^2$. To fully understand this effect, we conduct the systematical simulations on mild-size graphs varying in the number of nodes and the graph connectivity. Specifically, we study u$d$r-$n$, w$d$r-$n$, u$d$e-$n$ and w$d$e-$n$ with $d \in [3, 9]$ and $n \in [20, 100]$. For each setting, we generate 10 instances and use the average approximation ratio of them to evaluate the performance of QAOA$^2$. Here both $C^*_{\text{SDP}}$ and $C^*_{\text{asymp}}$ are too loose, so we use the value of cut searched by GW algorithm as $C^*$ to calculate the approximation ratio $r_{\mathcal{A}}$. As shown in Fig. 5(a), an evident observation is that QAOA$^2$ prefers denser graph than sparse graph. For example, QAOA$^2$ achieves an approximation ratio of 0.9488 on u9r-20 but only 0.5836 on u3r-100. Consequently, a reasonable conjecture is that the main contribution of cuts comes from the cuts inside subgraphs. In order to improve the performance of QAOA$^2$, one possible way is making the subgraphs as

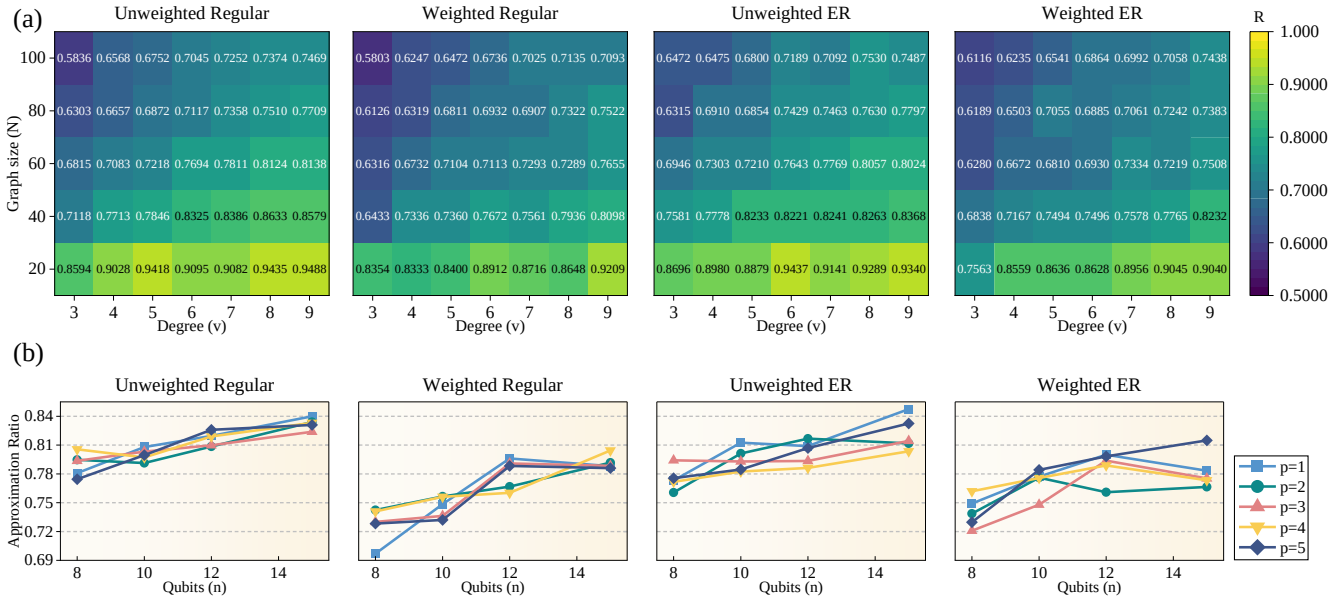


FIG. 5. (a) **Influence of graph density**. In heat map, x-axis represents the degree of vertices and the y-axis represents the size of graphs. The color of points indicates the approximation ratio of corresponding graph setting. Brighter color means higher ratio. (b) **QAOA$^2$ with different qubit size**. Fix graphs to 60-vertex and 9-degree. The performance grows as the qubit size grows from 8 to 15. Five types of curves represents the level of QAOA circuit from 1-5.

dense as possible. An alternative approach is adopting a better graph partition strategy such as the community detection method discussed in Appendix A.

Apart from the property of graph, the hyperparameter setting of QAOA$^2$, i.e., the qubit counts $n$ and the number of layers $p$, may also effect its performance. With this regard, we consider the setting of graphs with 60 vertices and 9 degree i.e. u9r-60, u9e-60, w9r-60, and w9e-60. The number of qubits $n$ is chosen in $\{8, 10, 12, 15\}$ and the level $p$ is chosen in $[1, 5]$. The results are collected across 10 instances for each setting. The achieved results are shown in Fig. 5(b). Specifically, under the measure of the averaged approximation ration, a deeper level $p$ slightly contribute much improvement. A concrete example is u9e-60, where the performance of QAOA$^2$ with $p = 4$ is inferior to the performance of QAOA$^2$ with $p = 3$. Nevertheless, the performance QAOA$^2$ can increase significantly with the larger subgraph size $n$. For example, when the level is specified to be $p = 1$, the approximation ratio of QAOA$^2$ is increased by 0.06 in u9r-60 and 0.1 in w9r-60 when $n$ improves from 8 to 14. These observations indicate that executing QAOA$^2$ on a large quantum system contributes to a better performance.

## V. DISCUSSION

In this study, we propose QAOA$^2$ that utilizes the structure of graphs and $\mathbb{Z}_2$ symmetry to solve large-scale MaxCut problems on small-scale quantum devices. We prove that a hierarchical scheme can be achieved via reformulated MaxCuts. The approximation ratio is always greater than 0.5. The numerical results show that the proposed QAOA$^2$ achieves comparable performance on 2000-vertex graphs against the best known classical algorithm. Moreover, numerical results indicates that QAOA$^2$ can attain better performance for denser graphs which are hard for conventional QAOAs. Our work sheds light on solving large-scale problems with potential advantages in NISQ era.

There are several important future research directions. First, it is crucial to design more instance-aware partition strategies to further improve the capabilities of QAOA$^2$. Furthermore, an intriguing direction is integrating QAOA$^2$ with with distributed variational quantum optimization techniques [53–55], which allows us to accelerate the optimization and understand the power of QAOA$^2$ on large-scale problems. Next, since the subgraphs are independent, the performance of QAOA$^2$ could be enhanced by employing advanced and problem-specific local QAOA$^2$ solvers [38, 40, 56]. Last, the concept of decomposing Hamiltonian by its symmetric property used in QAOA$^2$ can be generalized to boost other variational quantum algorithms. For example, in quantum chemistry, some proposals of variational eigensolvers have used the natural symmetry of some molecular to reduce the required number of qubits [57–59]. In quantum machine learning, the concept of decomposing Hamiltonian by its symmetric property can be leveraged to design powerful Hamiltonian-based quantum neural networks with some invariant properties [60, 61]. In this way, these QNNs can attain better convergence and generalization [62–66].

[1] Bernhard Korte and Jens Vygen. *Combinatorial Optimization: Theory and Algorithms*. Algorithms and Combinatorics. Springer-Verlag, Berlin Heidelberg, third edition, 2006.

[2] A. Juarna. Combinatorial Algorithms for Portfolio Optimization Problems – Case of Risk Moderate Investor. *J. Phys.: Conf. Ser.*, 820:012028, March 2017.

[3] Abdelkader Sbihi and Richard W. Eglese. Combinatorial optimization and Green Logistics. *4OR*, 5(2):99–116, July 2007.

[4] Michel X. Goemans and David P. Williamson. .879-approximation algorithms for MAX CUT and MAX 2SAT. In *Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing*, STOC '94, pages 422–431, New York, NY, USA, May 1994. Association for Computing Machinery.

[5] Michel X. Goemans and David P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *J. ACM*, 42(6):1115–1145, November 1995.

[6] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by Simulated Annealing. *Science*, 220(4598):671–680, May 1983.

[7] David S. Johnson. Local optimization and the Traveling Salesman Problem. In Michael S. Paterson, editor, *Automata, Languages and Programming*, Lecture Notes in Computer Science, pages 446–461, Berlin, Heidelberg, 1990. Springer.

[8] Michael Jünger, Gerhard Reinelt, and Giovanni Rinaldi. Chapter 4 The traveling salesman problem. In *Handbooks in Operations Research and Management Science*, volume 7 of *Network Models*, pages 225–330. Elsevier, January 1995.

[9] César Rego, Dorabela Gamboa, Fred Glover, and Colin Osterman. Traveling salesman problem heuristics: Leading methods, implementations and latest advances. *European Journal of Operational Research*, 211(3):427–441, June 2011.

[10] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine Learning for Combinatorial Optimization: A Methodological Tour d'Horizon. *arXiv:1811.06128 [cs, stat]*, March 2020.

[11] Nina Mazyavkina, Sergey Sviridov, Sergei Ivanov, and Evgeny Burnaev. Reinforcement learning for combinatorial optimization: A survey. *Computers & Operations Research*, 134:105400, October 2021.

[12] Quentin Cappart, Didier Chételat, Elias Khalil, Andrea Lodi, Christopher Morris, and Petar Veličković. Combinatorial optimization and reasoning with graph neural networks. *arXiv:2102.09544 [cs, math, stat]*, April 2021.

[13] Richard M. Karp. Reducibility among Combinatorial Problems. In Raymond E. Miller, James W. Thatcher, and Jean D. Bohlinger, editors, *Complexity of Computer Computations: Proceedings of a Symposium on the Complexity of Computer Computations, Held March 20–22, 1972, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, and Sponsored by the Office of Naval Research, Mathematics Program, IBM World Trade Corporation, and the IBM Research Mathematical Sciences Department*, The IBM Research Symposia Series, pages 85–103. Springer US, Boston, MA, 1972.

[14] P.W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134, November 1994.

[15] John Preskill. Quantum Computing in the NISQ era and beyond. *arXiv:1801.00862 [cond-mat, physics:quant-ph]*, July 2018.

[16] Han-Sen Zhong, Hui Wang, Yu-Hao Deng, Ming-Cheng Chen, Li-Chao Peng, Yi-Han Luo, Jian Qin, Dian Wu, Xing Ding, Yi Hu, Peng Hu, Xiao-Yan Yang, Wei-Jun Zhang, Hao Li, Yuxuan Li, Xiao Jiang, Lin Gan, Guangwen Yang, Lixing You, Zhen Wang, Li Li, Nai-Le Liu, Chao-Yang Lu, and Jian-Wei Pan. Quantum computational advantage using photons. *Science*, 370(6523):1460, December 2020.

[17] Frank Arute, Kunal Arya, Ryan Babbush, Dave Bacon, Joseph C. Bardin, Rami Barends, Rupak Biswas, Sergio Boixo, Fernando G. S. L. Brandao, David A. Buell, Brian Burkett, Yu Chen, Zijun Chen, Ben Chiaro, Roberto Collins, William Courtney, Andrew Dunsworth, Edward Farhi, Brooks Foxen, Austin Fowler, Craig Gidney, Marissa Giustina, Rob Graff, Keith Guerin, Steve Habegger, Matthew P. Harrigan, Michael J. Hartmann, Alan Ho, Markus Hoffmann, Trent Huang, Travis S. Humble, Sergei V. Isakov, Evan Jeffrey, Zhang Jiang, Dvir Kafri, Kostyantyn Kechedzhi, Julian Kelly, Paul V. Klimov, Sergey Knysh, Alexander Korotkov, Fedor Kostritsa, David Landhuis, Mike Lindmark, Erik Lucero, Dmitry Lyakh, Salvatore Mandrà, Jarrod R. McClean, Matthew McEwen, Anthony Megrant, Xiao Mi, Kristel Michielsen, Masoud Mohseni, Josh Mutus, Ofer Naaman, Matthew Neeley, Charles Neill, Murphy Yuezhen Niu, Eric Ostby, Andre Petukhov, John C. Platt, Chris Quintana, Eleanor G. Rieffel, Pedram Roushan, Nicholas C. Rubin, Daniel Sank, Kevin J. Satzinger, Vadim Smelyanskiy, Kevin J. Sung, Matthew D. Trevithick, Amit Vainsencher, Benjamin Villalonga, Theodore White, Z. Jamie Yao, Ping Yeh, Adam Zalcman, Hartmut Neven, and John M. Martinis. Quantum supremacy using a programmable superconducting processor. *Nature*, 574(7779):505–510, October 2019.

[18] Yulin Wu, Wan-Su Bao, Sirui Cao, Fusheng Chen, Ming-Cheng Chen, Xiawei Chen, Tung-Hsun Chung, Hui Deng, Yajie Du, Daojin Fan, Ming Gong, Cheng Guo, Chu Guo, Shaojun Guo, Lianchen Han, Linyin Hong, He-Liang Huang, Yong-Heng Huo, Liping Li, Na Li, Shaowei Li, Yuan Li, Futian Liang, Chun Lin, Jin Lin, Haoran Qian, Dan Qiao, Hao Rong, Hong Su, Lihua Sun, Liangyuan Wang, Shiyu Wang, Dachao Wu, Yu Xu, Kai Yan, Weifeng Yang, Yang Yang, Yangsen Ye, Jianghan Yin, Chong Ying, Jiale Yu, Chen Zha, Cha Zhang, Haibin Zhang, Kaili Zhang, Yiming Zhang, Han Zhao, Youwei Zhao, Liang Zhou, Qingling Zhu, Chao-Yang Lu, Cheng-Zhi Peng, Xiaobo Zhu, and Jian-Wei Pan. Strong quantum computational advantage using a superconducting quantum processor. *arXiv:2106.14734 [quant-ph]*, June 2021.

[19] M. Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nat*

*Rev Phys*, 3(9):625–644, September 2021.

[20] Marcello Benedetti, Erika Lloyd, Stefan Sack, and Mattia Fiorentini. Parameterized quantum circuits as machine learning models. *Quantum Sci. Technol.*, 4(4):043001, November 2019.

[21] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. A Quantum Approximate Optimization Algorithm. *arXiv:1411.4028 [quant-ph]*, November 2014.

[22] Edward Farhi and Aram W. Harrow. Quantum Supremacy through the Quantum Approximate Optimization Algorithm. *arXiv:1602.07674 [quant-ph]*, October 2019.

[23] G. G. Guerreschi and A. Y. Matsuura. QAOA for Max-Cut requires hundreds of qubits for quantum speed-up. *Sci Rep*, 9(1):6903, December 2019.

[24] Andrew Lucas. Ising formulations of many NP problems. *Front. Physics*, 2, 2014.

[25] Fred Glover, Gary Kochenberger, and Yu Du. A Tutorial on Formulating and Using QUBO Models. *arXiv:1811.11538 [quant-ph]*, November 2019.

[26] Ryan Hamerly, Takahiro Inagaki, Peter L. McMahon, Davide Venturelli, Alireza Marandi, Tatsuhiro Onodera, Edwin Ng, Carsten Langrock, Kensuke Inaba, Toshimori Honjo, Koji Enbutsu, Takeshi Umeki, Ryoichi Kasahara, Shoko Utsunomiya, Satoshi Kako, Ken-ichi Kawarabayashi, Robert L. Byer, Martin M. Fejer, Hideo Mabuchi, Dirk Englund, Eleanor Rieffel, Hiroki Takesue, and Yoshihisa Yamamoto. Experimental investigation of performance differences between Coherent Ising Machines and a quantum annealer. *Sci. Adv.*, 5(5):eaau0823, May 2019.

[27] G. Pagano, A. Bapat, P. Becker, K. S. Collins, A. De, P. W. Hess, H. B. Kaplan, A. Kyprianidis, W. L. Tan, C. Baldwin, L. T. Brady, A. Deshpande, F. Liu, S. Jordan, A. V. Gorshkov, and C. Monroe. Quantum Approximate Optimization of the Long-Range Ising Model with a Trapped-Ion Quantum Simulator. *Proc Natl Acad Sci USA*, 117(41):25396–25401, October 2020.

[28] Matthew P. Harrigan, Kevin J. Sung, Matthew Neeley, Kevin J. Satzinger, Frank Arute, Kunal Arya, Juan Atalaya, Joseph C. Bardin, Rami Barends, Sergio Boixo, Michael Broughton, Bob B. Buckley, David A. Buell, Brian Burkett, Nicholas Bushnell, Yu Chen, Zijun Chen, Ben Chiaro, Roberto Collins, William Courtney, Sean Demura, Andrew Dunsworth, Daniel Eppens, Austin Fowler, Brooks Foxen, Craig Gidney, Marissa Giustina, Rob Graff, Steve Habegger, Alan Ho, Sabrina Hong, Trent Huang, L. B. Ioffe, Sergei V. Isakov, Evan Jeffrey, Zhang Jiang, Cody Jones, Dvir Kafri, Kostyantyn Kechedzhi, Julian Kelly, Seon Kim, Paul V. Klimov, Alexander N. Korotkov, Fedor Kostritsa, David Landhuis, Pavel Laptev, Mike Lindmark, Martin Leib, Orion Martin, John M. Martinis, Jarrod R. McClean, Matt McEwen, Anthony Megrant, Xiao Mi, Masoud Mohseni, Wojciech Mruczkiewicz, Josh Mutus, Ofer Naaman, Charles Neill, Florian Neukart, Murphy Yuezhen Niu, Thomas E. O'Brien, Bryan O'Gorman, Eric Ostby, Andre Petukhov, Harald Putterman, Chris Quintana, Pedram Roushan, Nicholas C. Rubin, Daniel Sank, Andrea Skolik, Vadim Smelyanskiy, Doug Strain, Michael Streif, Marco Szalay, Amit Vainsencher, Theodore White, Z. Jamie Yao, Ping Yeh, Adam Zalcman, Leo Zhou, Hartmut Neven, Dave Bacon, Erik Lucero, Edward Farhi, and Ryan Babbush. Quantum Approximate Optimization of Non-Planar Graph Problems on a Planar Superconducting Processor. *Nat. Phys.*, 17(3):332–336, March 2021.

[29] Edward Farhi, Jeffrey Goldstone, Sam Gutmann, and Michael Sipser. Quantum Computation by Adiabatic Evolution. *arXiv:quant-ph/0001106*, January 2000.

[30] Edward Farhi, Jeffrey Goldstone, Sam Gutmann, Joshua Lapan, Andrew Lundgren, and Daniel Preda. A Quantum Adiabatic Evolution Algorithm Applied to Random Instances of an NP-Complete Problem. *Science*, 292(5516):472–475, April 2001.

[31] Leo Zhou, Sheng-Tao Wang, Soonwon Choi, Hannes Pichler, and Mikhail D. Lukin. Quantum Approximate Optimization Algorithm: Performance, Mechanism, and Implementation on Near-Term Devices. *Phys. Rev. X*, 10(2):021067, June 2020.

[32] Jonathan Wurtz and Peter J. Love. Counterdiabaticity and the quantum approximate optimization algorithm. *arXiv:2106.15645 [quant-ph]*, June 2021.

[33] Fernando G. S. L. Brandao, Michael Broughton, Edward Farhi, Sam Gutmann, and Hartmut Neven. For Fixed Control Parameters the Quantum Approximate Optimization Algorithm's Objective Function Value Concentrates for Typical Instances. *arXiv:1812.04170 [quant-ph]*, December 2018.

[34] Yunlong Yu, Chenfeng Cao, Carter Dewey, Xiang-Bin Wang, Nic Shannon, and Robert Joynt. Quantum Approximate Optimization Algorithm with Adaptive Bias Fields. *arXiv:2105.11946 [quant-ph]*, May 2021.

[35] Panagiotis Kl Barkoutsos, Giacomo Nannicini, Anton Robert, Ivano Tavernelli, and Stefan Woerner. Improving Variational Quantum Optimization using CVaR. *Quantum*, 4:256, April 2020.

[36] Sergey Bravyi, Alexander Kliesch, Robert Koenig, and Eugene Tang. Obstacles to State Preparation and Variational Optimization from Symmetry Protection. *Phys. Rev. Lett.*, 125(26):260505, December 2020.

[37] E. Campos, D. Rabinovich, V. Akshay, and J. Biamonte. Training Saturation in Layerwise Quantum Approximate Optimisation. *arXiv:2106.13814 [cond-mat, physics:quant-ph]*, June 2021.

[38] Linghua Zhu, Ho Lun Tang, George S. Barron, F. A. Calderon-Vargas, Nicholas J. Mayhall, Edwin Barnes, and Sophia E. Economou. An adaptive quantum approximate optimization algorithm for solving combinatorial problems on a quantum computer. *arXiv:2005.10258 [quant-ph]*, December 2020.

[39] Stuart Hadfield, Zhihui Wang, Bryan O'Gorman, Eleanor G. Rieffel, Davide Venturelli, and Rupak Biswas. From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz. *Algorithms*, 12(2):34, February 2019.

[40] Zhihui Wang, Nicholas C. Rubin, Jason M. Dominy, and Eleanor G. Rieffel. $XY$-mixers: Analytical and numerical results for QAOA. *Phys. Rev. A*, 101(1):012320, January 2020.

[41] Sepehr Ebadi, Alexander Keesling, Madelyn Cain, Tout T. Wang, Harry Levine, Dolev Bluvstein, Giulia Semeghini, Ahmed Omran, Jinguo Liu, Rhine Samajdar, Xiu-Zhe Luo, Beatrice Nash, Xun Gao, Boaz Barak, Edward Farhi, Subir Sachdev, Nathan Gemelke, Leo Zhou, Soonwon Choi, Hannes Pichler, Shengtao Wang, Markus Greiner, Vladan Vuletic, and Mikhail D. Lukin. Quantum Optimization of Maximum Independent Set using Rydberg Atom Arrays. *arXiv:2202.09372 [cond-mat,*

*physics:physics, physics:quant-ph]*, February 2022.

[42] Phillip C. Lotshaw, Thien Nguyen, Anthony Santana, Alexander McCaskey, Rebekah Herrman, James Ostrowski, George Siopsis, and Travis S. Humble. Scaling Quantum Approximate Optimization on Near-term Hardware. *arXiv:2201.02247 [quant-ph]*, January 2022.

[43] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9(1):4812, November 2018.

[44] Juneseo Lee, Alicia B. Magann, Herschel A. Rabitz, and Christian Arenz. Progress toward favorable landscapes in quantum combinatorial optimization. *Phys. Rev. A*, 104(3):032401, September 2021.

[45] Yuxuan Du, Min-Hsiu Hsieh, Tongliang Liu, Shan You, and Dacheng Tao. Learnability of quantum neural networks. *PRX Quantum*, 2(4):040337, 2021.

[46] Kaining Zhang, Min-Hsiu Hsieh, Liu Liu, and Dacheng Tao. Toward trainability of deep quantum neural networks. *arXiv preprint arXiv:2112.15002*, 2021.

[47] Jeffrey Marshall, Filip Wudarski, Stuart Hadfield, and Tad Hogg. Characterizing local noise in qaoa circuits. *IOP SciNotes*, 1(2):025208, 2020.

[48] Junde Li, Mahabubul Alam, and Swaroop Ghosh. Large-scale Quantum Approximate Optimization via Divide-and-Conquer. *arXiv:2102.13288 [quant-ph]*, February 2021.

[49] Vishwanathan Akshay, Daniil Rabinovich, Ernesto Campos, and Jacob Biamonte. Parameter concentrations in quantum approximate optimization. *Physical Review A*, 104(1):L010401, 2021.

[50] The approach proposed by [48] breaks one graph into two subgraphs sharing common nodes. To sample a good candidate solution, the local solution of these common nodes should be exactly overlapped. In this respect, the sample complexity of their approach grows with number of common nodes which makes it harder to sample a good candidate solution.

[51] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, Cambridge, 2010.

[52] Amir Dembo, Andrea Montanari, and Subhabrata Sen. Extremal Cuts of Sparse Random Graphs. *Ann. Probab.*, 45(2), March 2017.

[53] Harry Buhrman and Hein Röhrig. Distributed Quantum Computing. In Branislav Rovan and Peter Vojtáš, editors, *Mathematical Foundations of Computer Science 2003*, Lecture Notes in Computer Science, pages 1–20, Berlin, Heidelberg, 2003. Springer.

[54] Daniele Cuomo, Marcello Caleffi, and Angela Sara Cacciapuoti. Towards a Distributed Quantum Computing Ecosystem. *IET Quantum Communication*, 1(1):3–8, July 2020.

[55] Y. Du, Y. Qian, X. Wu, and D. Tao. Accelerating variational quantum algorithms with multiple quantum processors. *IEEE Transactions on Quantum Engineering*, (01):1–1, may 2022.

[56] Yuxuan Du, Tao Huang, Shan You, Min-Hsiu Hsieh, and Dacheng Tao. Quantum circuit architecture search for variational quantum algorithms. *npj Quantum Information*, 8, 2022.

[57] Sergey Bravyi, Jay M. Gambetta, Antonio Mezzacapo, and Kristan Temme. Tapering off qubits to simulate fermionic Hamiltonians. *arXiv:1701.08213 [quant-ph]*, January 2017.

[58] Jin-Guo Liu, Yi-Hong Zhang, Yuan Wan, and Lei Wang. Variational Quantum Eigensolver with Fewer Qubits. *Phys. Rev. Research*, 1(2):023025, September 2019.

[59] Changsu Cao, Jiaqi Hu, Wengang Zhang, Xusheng Xu, Dechin Chen, Fan Yu, Jun Li, Hanshi Hu, Dingshun Lv, and Man-Hong Yung. Towards a Larger Molecular Simulation on the Quantum Computer: Up to 28 Qubits Systems Accelerated by Point Group Symmetry. *arXiv:2109.02110 [quant-ph]*, September 2021.

[60] Johannes Jakob Meyer, Marian Mularski, Elies Gil-Fuster, Antonio Anna Mele, Francesco Arzani, Alissa Wilms, and Jens Eisert. Exploiting symmetry in variational quantum machine learning. *arXiv preprint arXiv:2205.06217*, 2022.

[61] Andrea Skolik, Michele Cattelan, Sheir Yarkoni, Thomas Bäck, and Vedran Dunjko. Equivariant quantum circuits for learning on weighted graphs. *arXiv preprint arXiv:2205.06109*, 2022.

[62] Junyu Liu, Khadijeh Najafi, Kunal Sharma, Francesco Tacchino, Liang Jiang, and Antonio Mezzacapo. An analytic theory for the dynamics of wide quantum neural networks, 2022. arXiv:2203.16711v1.

[63] Yuxuan Du, Zhuozhuo Tu, Xiao Yuan, and Dacheng Tao. Efficient measure for the expressivity of variational quantum algorithms. *Physical Review Letters*, 128(8):080506, 2022.

[64] Hsin-Yuan Huang, Richard Kueng, and John Preskill. Information-theoretic bounds on quantum advantage in machine learning. *Physical Review Letters*, 126(19):190505, 2021.

[65] Amira Abbas, David Sutter, Christa Zoufal, Aurélien Lucchi, Alessio Figalli, and Stefan Woerner. The power of quantum neural networks. *Nature Computational Science*, 1(6):403–409, 2021.

[66] Yuxuan Du, Zhuozhuo Tu, Bujiao Wu, Xiao Yuan, and Dacheng Tao. Theory of quantum generative learning models with maximum mean discrepancy. *arXiv preprint arXiv:2205.04730*, 2022.

[67] M. E. J. Newman and M. Girvan. Finding and evaluating community structure in networks. *Phys. Rev. E*, 69(2):026113, February 2004.

[68] Aaron Clauset, M. E. J. Newman, and Cristopher Moore. Finding community structure in very large networks. *arXiv:cond-mat/0408187*, August 2004.

[69] M. E. J. Newman. Fast algorithm for detecting community structure in networks. *Phys. Rev. E*, 69(6):066133, June 2004.

[70] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *J. Stat. Mech.*, 2008(10):P10008, October 2008.

[71] Pranav Gokhale, Olivia Angiuli, Yongshan Ding, Kaiwen Gui, Teague Tomesh, Martin Suchara, Margaret Martonosi, and Frederic T. Chong. Minimizing State Preparations in Variational Quantum Eigensolver by Partitioning into Commuting Families. *arXiv:1907.13623 [quant-ph]*, July 2019.

[72] Vladyslav Verteletskyi, Tzu-Ching Yen, and Artur F. Izmaylov. Measurement Optimization in the Variational Quantum Eigensolver Using a Minimum Clique Cover. *J. Chem. Phys.*, 152(12):124114, March 2020.

[73] Yu Zhang, Lukasz Cincio, Christian F. A. Negre, Piotr Czarnik, Patrick Coles, Petr M. Anisimov, Susan M.

Mniszewski, Sergei Tretiak, and Pavel A. Dub. Variational Quantum Eigensolver with Reduced Circuit Complexity. *arXiv:2106.07619 [quant-ph]*, June 2021.

[74] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, M. Sohaib Alam, Shahnawaz Ahmed, Juan Miguel Arrazola, Carsten Blank, Alain Delgado, Soran Jahangiri, Keri McKiernan, Johannes Jakob Meyer, Zeyue Niu, Antal Száva, and Nathan Killoran. PennyLane: Automatic differentiation of hybrid quantum-classical computations. *arXiv:1811.04968 [physics, physics:quant-ph]*, February 2020.

[75] Steven Diamond and Stephen Boyd. CVXPY: A Python-Embedded Modeling Language for Convex Optimization. *arXiv:1603.00943 [math]*, June 2016.

[76] Brendan O'Donoghue, Eric Chu, Neal Parikh, and Stephen Boyd. Conic Optimization via Operator Splitting and Homogeneous Self-Dual Embedding. *J Optim Theory Appl*, 169(3):1042–1068, June 2016.

[77] https://github.com/networkx/networkx.

# Supplementary Material: "QAOA-in-QAOA: solving large-scale MaxCut problems on small quantum machines"

The organization of Supplementary Material is as follows. In Appendix A, we present how advanced partitioned methods can further improve the performance of QAOA$^2$. In Appendix B, we provide the proof of Theorem 1. Then in Appendix C, we demonstrate the proof of Theorem 2. Subsequently, we discuss how QAOA$^2$ relates to the Hamiltonian splitting method ind Appendix D. Last, in Appendix E, we exhibit the omitted simulation details and more simulation results of QAOA$^2$.

## Appendix A: Graph partitioning

A crucial step in QAOA$^2$ is partitioning graph into subgraphs. We note that the way of partition is diverse. Here we present two possible partition methods, i.e., random partition and community detection based partition. We leave the design of more advanced partition methods as the future work.

**Random partition.** The algorithmic implementation of the random partition is as follows. Given the number of qubits $n$ and a graph with size $N$, random partition successively samples $n$ vertices as a subgraph without replacement until all $\lceil N/n \rceil$ subgraphs are collected.

We remark that for dense graphs, random partition promises a good learning performance of QAOA$^2$ since the probability of an edge existing between arbitrary two vertex is high. In contrast, for graphs whose expected vertex degree is low, random partition may lead to an inferior performance. This is because the collected subgraphs may contain few edges and most edges remaining between subgraphs.

**Community detection based partition.** According to the above explanations, a natural idea to enhance the power of QAOA$^2$ is to maintain as many edges inside each subgraph as possible, which in turn suppresses the error incurred by partition. With this regard, we introduce modularity [67] as a measure of the quality of partitioning. A mathematical definition of modularity is

$$Q = \frac{1}{2m} \sum_{ij} \left[ A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j), \tag{A1}$$

where $m$ is the sum of weights of all edges, $A_{ij}$ is the element of adjacency matrix, $k_i$ is the sum of weights of edges connected to vertex $i$, and $c_i$ is the community that the vertex $i$ is assigned. Intuitively, the term $\frac{k_i k_j}{2m}$ indicates the probability of edge existing between $i$ and $j$ in a randomized graph. When the fraction of edges within communities equals to a randomized graph, the quantity $Q$ will be zero. When $Q > 0.3$, it indicates a significant community structure [68].

QAOA$^2$ pursues a high modularity $Q$, where the connectivity in subgraphs is dense but the connectivity between different subgraphs is sparse. An algorithm searching for high modularity partition is referred to as *community detection algorithm*, which regards subgraphs as communities. Several algorithms have been proposed to maximize modularity and to find community structure in graphs [67–70]. Here we consider the greedy modularity maximization algorithm [68]. In particular, starting with each vertex being only member of its own community, the algorithm joins the pair of communities that increases the modularity the most. This procedure is continuously conducted until no such pair exists or the termination condition is met.

We benchmark the performance of QAOA$^2$ with different partition methods in Appendix E.

## Appendix B: Proof of Theorem 1

*Proof of Theorem 1.* Although $\boldsymbol{x}_i$ and $\bar{\boldsymbol{x}}_i$ yield the same local objective value (i.e., $C_i(\boldsymbol{x}_i) = C_i(\bar{\boldsymbol{x}}_i)$), they may lead to a distinct objective value (i.e., $C(\boldsymbol{x}_1 ... \boldsymbol{x}_i ... \boldsymbol{x}_h) \neq C(\boldsymbol{x}_1 ... \bar{\boldsymbol{x}}_i ... \boldsymbol{x}_h)$), because of the connection between $\mathsf{G}_i$ and other subgraphs $\{\mathsf{G}_{i'}\}_{i' \neq i}$. Considering that there are in total $2^h$ candidates in $\mathcal{Z}$ in Eq. (5), our goal here is formulating an equivalent objective function that finds the target bitstring satisfying $\hat{\boldsymbol{z}} = \arg\min_{\boldsymbol{z} \in \mathcal{Z}} C(\boldsymbol{z})$ using NISQ devices.

Considering two neighboring subgraphs $\mathsf{G}_i$ and $\mathsf{G}_j$, we denote $\boldsymbol{s}_i \in \{+1, -1\}$ as flipping indicator in the sense that $\boldsymbol{s}_i = -1$ flips $\boldsymbol{x}_i$ to be $\bar{\boldsymbol{x}}_i$ and $\boldsymbol{s}_i = +1$ keeps $\boldsymbol{x}_i$ unchanged. When two subgraphs are synchronous, i.e. $\boldsymbol{s}_i = \boldsymbol{s}_j$, the inter-cut between them is retained and the inter-cut size is

$$w_{ij}^{\text{sync}} = \sum_u \sum_v \frac{1}{2} (1 - \boldsymbol{x}_i^{(u)} \boldsymbol{x}_j^{(v)}) W_{iu,jv}, \tag{B1}$$

where $\boldsymbol{x}_i^{(u)}$ denotes the $u$-th bit in local solution bitstring $\boldsymbol{x}_i$ with $u \in [|\mathsf{V}_i|]$ and $W_{iu,jv}$ is the weight of edge in $\mathsf{G}$ corresponding to two vertices $\boldsymbol{x}_i^{(u)}$ and $\boldsymbol{x}_j^{(v)}$. Note that here we define $\boldsymbol{x}_i \in \{+1,-1\}^{|V_i|}$, the coefficient $\frac{1}{2}$ acts as a standardization term to make sure $w_{ij}^{\text{sync}}$ is the sum of cut edges between $\mathsf{G}_i$ and $\mathsf{G}_j$ when adopting $\boldsymbol{x}_i, \boldsymbol{x}_j$ (or $\bar{\boldsymbol{x}}_i, \bar{\boldsymbol{x}}_j$). When two subgraphs are asynchronous, i.e., $\boldsymbol{s}_i = -\boldsymbol{s}_j$, the inter-cut takes the form

$$\begin{aligned} w_{ij}^{\text{async}} &= \sum_u \sum_v \frac{1}{2}(1 - \boldsymbol{x}_i^{(u)} \bar{\boldsymbol{x}}_j^{(v)}) W_{iu,jv} \\ &= \sum_u \sum_v \frac{1}{2}(1 + \boldsymbol{x}_i^{(u)} \boldsymbol{x}_j^{(v)}) W_{iu,jv}, \end{aligned} \tag{B2}$$

where the second equality uses the relationship $\bar{\boldsymbol{x}}_j = -\boldsymbol{x}_j$.

Let $\boldsymbol{s} = (\boldsymbol{s}_1, \boldsymbol{s}_2, \ldots, \boldsymbol{s}_h) \in \{+1,-1\}^h$ be an $h$-length bitstring as the indicator for the selection of $\{\boldsymbol{x}_i, \bar{\boldsymbol{x}}_i\}$ to form $\boldsymbol{z}$. Define $w'_{ij} = w_{ij}^{\text{async}} - w_{ij}^{\text{sync}}$, $c = \sum_{i=1}^h C_i(\boldsymbol{x}_i) + \sum_{i<j} \frac{1}{2}(w_{ij}^{\text{sync}} + w_{ij}^{\text{async}})$, and $C_i(\boldsymbol{x}_i)$ as the local cut size of $\mathsf{G}_i$. For $\forall \boldsymbol{z} \in \mathcal{Z}$, the objective value $C(\boldsymbol{z})$ in Eq. (5) yields

$$\begin{aligned} & C(\boldsymbol{z}) \\ =& C(\boldsymbol{s}_1\boldsymbol{x}_1, \ldots, \boldsymbol{s}_h\boldsymbol{x}_h) \\ =& \sum_{i<j} \sum_u \sum_v \frac{1}{2}(1 - \boldsymbol{s}_i \boldsymbol{x}_i^{(u)} \boldsymbol{s}_j \boldsymbol{x}_j^{(v)}) W_{iu,jv} + \sum_i C_i(\boldsymbol{x}_i) \\ =& \sum_{i<j} \frac{1}{2} \left( -\boldsymbol{s}_i \boldsymbol{s}_j \sum_u \sum_v W_{iu,jv} \boldsymbol{x}_i^{(u)} \boldsymbol{x}_j^{(v)} + \sum_u \sum_v W_{iu,jv} \right) \\ & + \sum_i C_i(\boldsymbol{x}_i) \\ =& \sum_{i<j} \frac{1}{2} \left[ (w_{ij}^{\text{sync}} + w_{ij}^{\text{async}}) + (w_{ij}^{\text{sync}} - w_{ij}^{\text{async}}) \boldsymbol{s}_i \boldsymbol{s}_j \right] \\ & + \sum_i C_i(\boldsymbol{x}_i) \\ =& -\frac{1}{2} \sum_{i<j} w'_{ij} \boldsymbol{s}_i \boldsymbol{s}_j + c, \end{aligned}$$

where the second equality consists of two summations, i.e., the first term is the sum of inter cut between each pair of subgraphs and the second term is the sum of local cut inside subgraphs (which is not influenced by $\boldsymbol{s}$), the last second equality uses $w_{ij}^{\text{async}} - w_{ij}^{\text{sync}} = \sum_u \sum_v W_{iu,jv} \boldsymbol{x}_i^{(u)} \boldsymbol{x}_j^{(v)}$ and $w_{ij}^{\text{async}} + w_{ij}^{\text{sync}} = \sum_u \sum_v W_{iu,jv}$.

Define $C'(\boldsymbol{s}) = c - \frac{1}{2} \sum_{i<j} w'_{ij} \boldsymbol{s}_i \boldsymbol{s}_j$. Since all local cuts $\{C_i(\boldsymbol{x}_i)\}$ are fixed, we have

$$\max_{\boldsymbol{s} \in \{+1,-1\}^h} C'(\boldsymbol{s}) = \max_{\boldsymbol{z} \in \mathcal{Z}} C(\boldsymbol{z}). \tag{B3}$$

We find a good merging by optimizing the above. □

In most cases, optimizing merging of local solutions will improve the value of cut. However, if the local solutions happens to be in a good order, one can merge them without further optimization. We show the effect of merging optimization through experiments in Appendix E.

## Appendix C: Proof of Theorem 2

*Proof of Theorem 2.* We follow notations defined in Appendix B to prove Theorem 2. Suppose that $\mathsf{G}$ is partitioned into $h$ subgraphs $\mathsf{G}_1, \ldots, \mathsf{G}_h$, we can divide edges into two parts $\mathsf{E} = \mathsf{E}^{\text{inner}} \bigcup \mathsf{E}^{\text{inter}}$ where $\mathsf{E}^{\text{inner}} = \bigcup_{i=1}^h \mathsf{E}_i$ denotes the set of edges inside all subgraphs and $\mathsf{E}^{\text{inter}} = \bigcup_{1 \le i < j \le h} \mathsf{E}_{ij}$ denotes the set of edges between subgraphs. Here the

weight of edge $e \in \mathsf{E}$ is denoted by $w(e)$. Then we have

$$\begin{aligned}\sum_{e\in\mathsf{E}} w(e) &= \sum_{e\in\mathsf{E}^{\text{inner}}} w(e) + \sum_{e\in\mathsf{E}^{\text{inter}}} w(e) \\ &= \sum_{e\in\bigcup_{i=1}^{h}\mathsf{E}_i} w(e) + \sum_{e\in\bigcup_{1\le i<j\le h}\mathsf{E}_{ij}} w(e) \\ &= \sum_{i=1}^{h}\sum_{e\in\mathsf{E}_i} w(e) + \sum_{1\le i<j\le h}\sum_{e\in\mathsf{E}_{ij}} w(e).\end{aligned} \tag{C1}$$

When optimizing subgraphs, we can use any MaxCut solvers to return a set of local solutions $\{\boldsymbol{x}_i\}_{i=1}^{h}$ such that the cut value is greater than half of the sum of edge weights for all subgraphs $\{\mathsf{G}_i\}_{i=1}^{h}$. Mathematically, the sum of edge weight for each subgraph satisfies

$$C_i(\boldsymbol{x}_i) \ge \frac{1}{2}\sum_{e\in\mathsf{E}_i} w(e) \tag{C2}$$

and the sum of edge weight for all subgraphs yields

$$C^{\text{inner}} = \sum_{i=1}^{h} C_i(\boldsymbol{x}_i) \ge \frac{1}{2}\sum_{e\in\bigcup_{i=1}^{h}\mathsf{E}_i} w(e). \tag{C3}$$

The above result means that we can always at least obtain half of the sum of edge weight edges inside $h$ subgraphs.

Combining Eq. (C1) and Eq. (C3), an observation is that if QAOA$^2$ outputs a solution $\boldsymbol{s} = \{\boldsymbol{s}_i\}_{i=1}^{h}$ such that the cut value of intra-subgraphs achieves at least half of the second term $\sum_{1\le i<j\le h}\sum_{e\in\mathsf{E}_{ij}} w(e)$ in Eq. (C1), then the total cut value for the whole graph is greater than $\frac{1}{2}\sum_{e\in\mathsf{E}} w(e)$. Recall the terms $w_{ij}^{\text{async}}$ and $w_{ij}^{\text{sync}}$ defined in Theorem 1, we have

$$\begin{aligned}\sum_{e\in\mathsf{E}^{\text{inter}}} w(e) &= \sum_{1\le i<j\le h}\sum_{e\in\mathsf{E}_{ij}} w(e) \\ &= \sum_{1\le i<j\le h}\sum_{u}\sum_{v} W_{iu,jv} \\ &= \sum_{1\le i<j\le h} w_{ij}^{\text{async}} + w_{ij}^{\text{sync}} \\ &= \sum_{1\le i<j\le h} 2w_{ij}^{\text{sync}} + (w_{ij}^{\text{async}} - w_{ij}^{\text{sync}}) \\ &= \sum_{1\le i<j\le h} 2w_{ij}^{\text{sync}} + w'_{ij} \\ &= \sum_{1\le i<j\le h} 2w_{ij}^{\text{sync}} + \sum_{1\le i<j\le h} w'_{ij}.\end{aligned} \tag{C4}$$

Note that the reformulated MaxCut in Theorem 1 in QAOA$^2$ is

$$\begin{aligned}C'(\boldsymbol{s}) &= \sum_{i<j}\frac{1}{2}\left[(w_{ij}^{\text{sync}} + w_{ij}^{\text{async}}) + (w_{ij}^{\text{sync}} - w_{ij}^{\text{async}})\boldsymbol{s}_i\boldsymbol{s}_j\right] \\ &+ \sum_{i} C_i(\boldsymbol{x}_i) \\ &= \sum_{i<j} w_{ij}^{\text{sync}} + \sum_{i<j}(w_{ij}^{\text{async}} - w_{ij}^{\text{sync}})(1 - \boldsymbol{s}_i\boldsymbol{s}_j) \\ &+ \sum_{i} C_i(\boldsymbol{x}_i) \\ &= \sum_{i<j} w'_{ij}(1 - \boldsymbol{s}_i\boldsymbol{s}_j) + \sum_{i<j} w_{ij}^{\text{sync}} + C^{\text{inner}}\end{aligned}$$

where $\sum_{i<j} w'_{ij}(1 - \boldsymbol{s}_i \boldsymbol{s}_j)$ is optimized by the MaxCut solver and at least half of $\sum_{i<j} w'_{ij}$ is cut. Let $\boldsymbol{s}^*$ be the estimated solution, we have

$$\begin{aligned} C'(\boldsymbol{s}^*) &\geq \frac{1}{2}\sum_{i<j} w'_{ij} + \sum_{i<j} w_{ij}^{\text{sync}} + C^{\text{inner}} \\ &= \frac{1}{2}\sum_{e\in\mathsf{E}^{\text{inter}}} w(e) + C^{\text{inner}} \\ &\geq \frac{1}{2}\sum_{e\in\mathsf{E}^{\text{inter}}} w(e) + \frac{1}{2}\sum_{e\in\mathsf{E}^{\text{inner}}} w(e) \\ &= \frac{1}{2}\sum_{e\in\mathsf{E}} w(e) \end{aligned} \tag{C5}$$

where the first inequality uses the result of reformulated MaxCut, the first equality uses the result of Eq. (C3), the second inequality uses the result of Eq. (C4), and the last equality uses Eq. (C1). □

We end this section by illustrating when the lower bound is achieved, Consider the example of a four-vertex unweighted ring where $\mathsf{V} = \{1, 2, 3, 4\}$ and $\mathsf{E} = \{(1, 2), (2, 3), (3, 4), (1, 4)\}$. Suppose we partition it into two subgraphs $\mathsf{G}_1$ and $\mathsf{G}_2$ with $\mathsf{V}_1 = \{1, 3\}$ and $\mathsf{V}_2 = \{2, 4\}$. So further if one of the local solutions is $(+1, -1)$, the final cut is 2 and the ratio is $\frac{1}{2}$ no matter how the global solution is merged. A smarter partition will be $\mathsf{V}_1 = \{1, 2\}$ and $\mathsf{V}_2 = \{3, 4\}$ where all edges are cut eventually. This case shows that the lower bound can be attained in worst case and effective partition strategy can alleviate this issue.

## Appendix D: Relation with Hamiltonian splitting

Many quantum computing tasks such as QAOA or VQE aims to find an eigenstate (ground state or most excited state) corresponding to a target eigenvalue of a given Hamiltonian $H$. We prepare a quantum state $|\psi(\theta)\rangle$ on parameterized circuit and measure it with $H$. Then we feed $\langle\psi(\theta)|H|\psi(\theta)\rangle$ to an optimizer and update the parameters. To measure states on qubit quantum computers, this Hamiltonian is represented in terms of Pauli words and each Pauli word can be written as tensor product of Pauli matrices i.e. $H = \sum_k \alpha_k P_k$. Thus we have

$$\langle\psi(\theta)|H|\psi(\theta)\rangle = \sum_k \alpha_k \langle\psi(\theta)|P_k|\psi(\theta)\rangle. \tag{D1}$$

This allows us to measure each Pauli words individually and add them together. In QAOA for MaxCut problem, $H$ takes the form of 2 where all Pauli words are tensor products of two Pauli-Z matrices corresponding to edges in problem graph.

In general, an arbitrary Hamiltonian of $n$-qubit has $O(4^n)$ Pauli words so the query of circuit grows exponentially with qubit counts if we simply measure one Pauli word at a time. Note that two observables can be measured simultaneously if they are commutable. Moreover, if a group of pairwise commuting observables share the same eigenbasis that diagonalizes them all simultaneously, they can be also measured on the same prepared state. For example in MaxCut problem, all Pauli words share the same eigenbasis (computational basis) so we don't need to measure $O(n^2)$ terms individually but only once. One method to reduce circuit query is to split Hamiltonian into several clusters and each cluster is a commuting group of Pauli words we mentioned above. The less number of clusters, the less of circuit query. Recent researches mapped Hamiltonian splitting task into MinCliqueCover problem [71, 72]. Besides, additional speedup can be introduced by distributed quantum computing on multiple untangled quantum computers [55].

Nevertheless, this does not reduce the required qubit counts on quantum computer. In order to reduce qubit counts, we need to ensure that any two clusters do not share common qubit i.e. Hamiltonian of each cluster cannot be used to measure the same qubit. Since there are always Pauli words with small coefficients which play little role in final $\langle\psi(\theta)|H|\psi(\theta)\rangle$, one can pretend they don't exists when constructing circuit thus partition qubits into several parts. If so, we can built several small and independent circuit and measure each cluster on distributed quantum computers with less qubits. The partitioning may follow the property of primal problem such as graph weight or the mutual information between clusters [73]. To minimize the performance loss introduced by partitioning, one can use dressed Hamiltonian in measuring [73] or include a fixing step as we discussed in Appendix B.

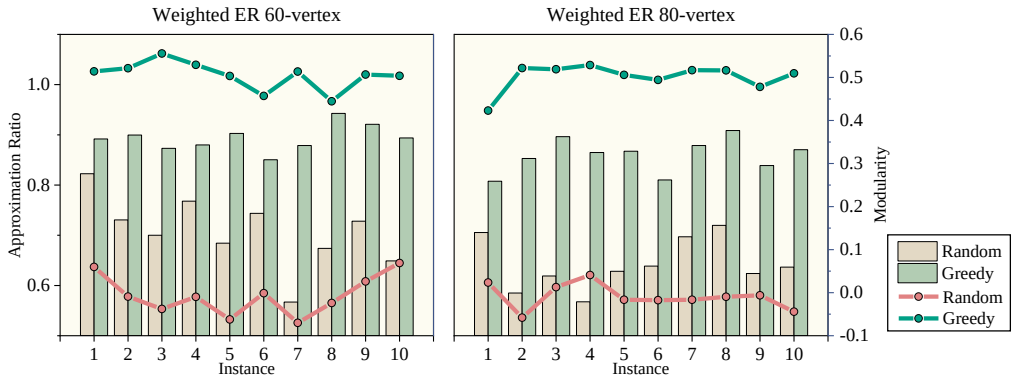


FIG. 6. **Results between random and greedy partition**. The yellow bars and green bars refer to approximation ratios of two partition strategies. The dotted line refers to the modularity of partition of each graph, which measures the quality of partitioning.

## Appendix E: Details of numerical simulations

### 1. Implementation details of QAOA$^2$

**Implementation details of QAOA$^2$.** The QAOA used in QAOA$^2$ are implemented by Pennylane [74]. In optimization, QAOA$^2$ adopts the vanilla gradient descent method to update the trainable parameters of QAOAs in which the learning rate is set as 0.01. The number of shots of each circuit is set as 1000 to approximate the expectation value of the measurements in calculating gradients. In the process of sampling solutions, QAOA$^2$ runs the optimized circuit for 1000 times to return the same number of bitstrings as solution candidates. To be more specific, in each run, all $n$ qubits are measured by the computational basis along the $Z$ direction and the measured result of each qubit is either +1 or -1. Thus an $n$-dimensional bitstring $z \in \{+1, -1\}^n$ is sampled in one query of the circuit. We collect 1000 such bitstrings as candidates and select the one as solution whose corresponding eigenvalue is the smallest among the candidates.

The source code of QAOA$^2$ is available at the Github Repository `https://github.com/ZeddTheGoat/QAOA_in_QAQA`.

**GW algorithm.** GW maps the primal integer programming to continuous vector space and optimizes it with semidefinte programming. The binary solution is obtained by projecting vectors to a plane. Our SDP solver is implemented by CVXPY [75], which uses SCS (Splitting Conic Solver) [76] following conventions. Here we adopt the default parameters in executing GW, where the max number of iterations is 2500, the convergence tolerance is $1e^{-4}$, the relaxation parameter is 1.8, and the balance between minimizing primal and dual residual is 5.0. After optimization, the solution is projected to 100 random vectors and rounded to bitstrings.

### 2. More simulation results of QAOA$^2$

**Performance of QAOA$^2$ with the advanced partition methods.** We investigate how the advanced partition method, i.e., the greedy community detection introduced in Appendix B, effects the performance of QAOA$^2$. To do so, we apply QAOA$^2$ to two types of graphs, i.e., w3e-60 and w3e-80. To collect the statistical results, we generate 10 instances for each setting. The allowable number of qubits is $n < 15$. The implementation of the community detection method follows the algorithm proposed in [68], which is realized by NetworkX library [77].

The simulation results are showin in Fig. 6, where the performance of QAOA$^2$ is dramatically increased when the random parition is replaced by the community detection. Specifically, for the 7-th instance of w3e-60 and the 4-th instance of w30e-80, the approximation ratio is increased by more than 0.3 compared to the random partition. Uder the measure of the modularities defined in Eq. A1, the achieved results of QAOA$^2$ with the community detection strategy are all above 0.4. This indicates good partition, which implies that most edges are kept within subgraphs. By contrast, the modularities of QAOA$^2$ with the random partition strategy are about 0. These observations accord with our conjecture in main text that the cut mainly comes from within subgraphs. The advanced partition strategy, e.g., the community detection, ensures the subgraphs as dense as possible and the number of edges left between subgraphs is minimized.

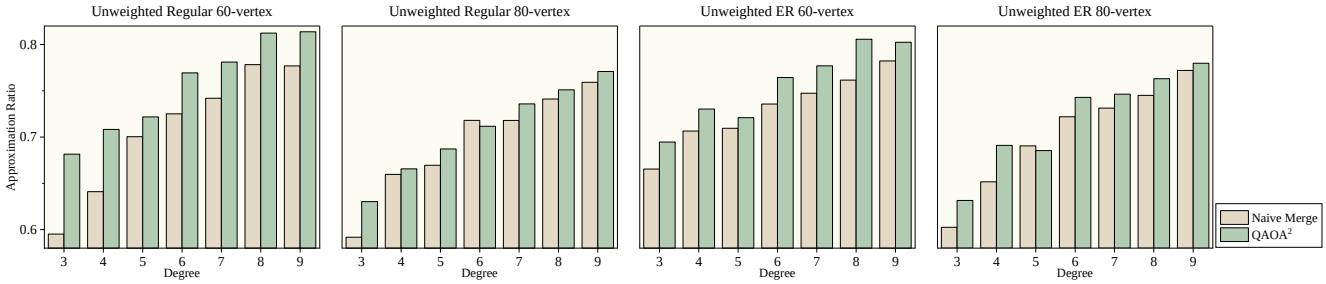


FIG. 7. **Results between Naive merging and QAOA$^2$.** The yellow bars represents the approximation ratio of QAOA$^2$, where solutions are merged naively. The green bars represents the approximation ratio of QAOA$^2$ introduced in the main text.

**Effect of merging optimization.** We next elucidate the importance of recasting the merging process as a new MaxCut problem used in QAOA$^2$. Particularly, we conduct an ablation study by evaluating the performance of QAOA$^2$ when the merging process is replaced by a naive heuristic, i.e., flipping non-local solutions as $\boldsymbol{s} = \mathbf{1}$. To do so, we compare these two merging methods on u$d$r-60, u$d$r-80, u$d$e-60 and u$d$e-80 with $d \in [3, 9]$ and 10 instances per setting. The number of qubits is set as $n < 10$. The collected results are shown in Fig. 7. For almost all settings, QAOA$^2$ outperforms the naive approach, except for u6r-80 and u5e-80. One possible reason is that local solutions is already good enough.